

# Context Diagrams

Bonita Sharif

School of Computing

University of Nebraska - Lincoln

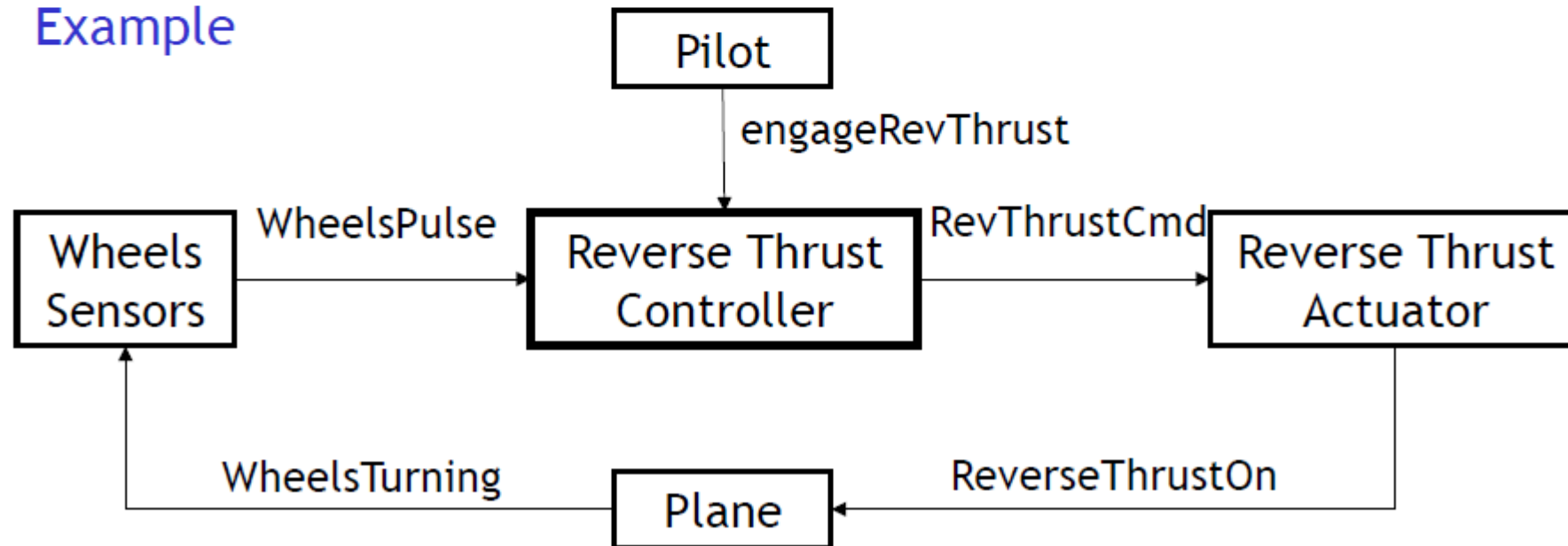
## Motivation

- When designing the Machine, it is essential to
  - understand the context (the World) in which the Machine operates
  - clarify the boundaries between the Machine and the World
- Failing to understand the context can lead to
  - failing to identify external entities with which machine must interact
  - making wrong assumptions about scope of what to build
  - making wrong assumptions about external entities

# Context Diagram

Definition: An **context diagram** is a model showing how the Machine and other agents in the World are connected through shared phenomena.

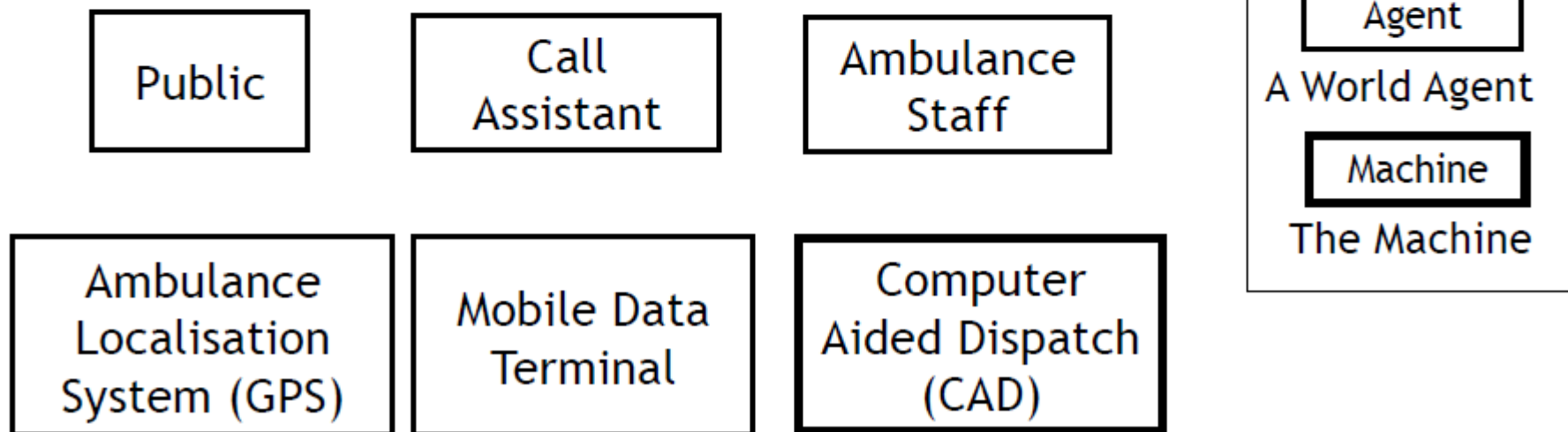
## Example



# Agents

Definition: An **agent** is an entity that controls some World phenomena. Agents can be human, human organisations, software, or devices. The Machine is also an agent.

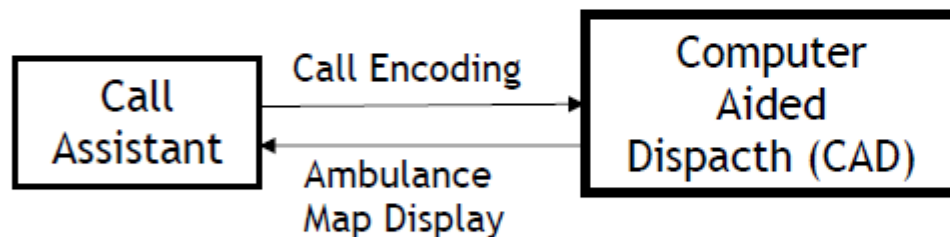
## Examples



# Interfaces

Definition: An **interface** is a boundary where two agents connect through a set of shared phenomena.

## Examples



## Legend

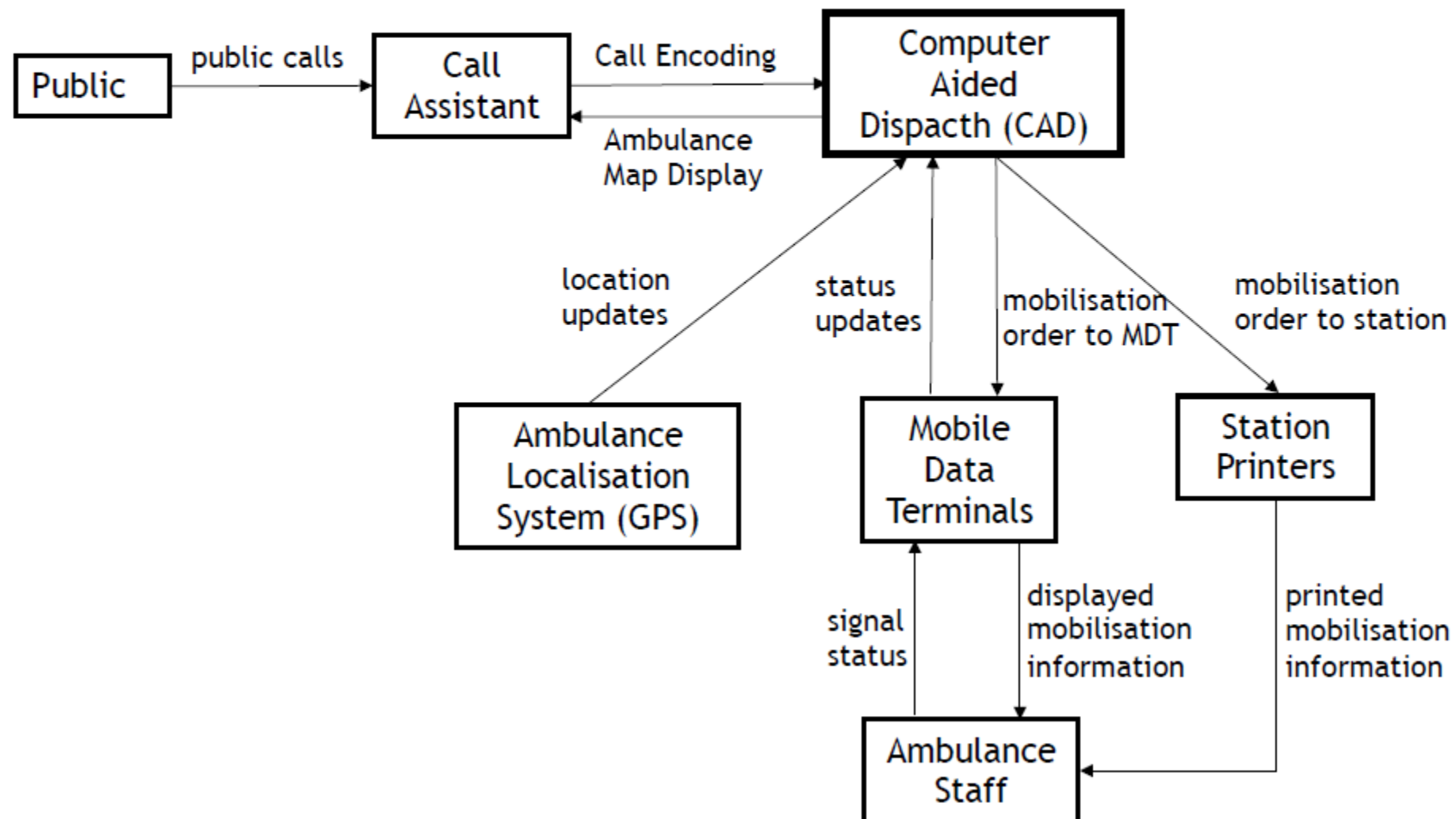


An interface where all phenomena are controlled by A and monitored by B

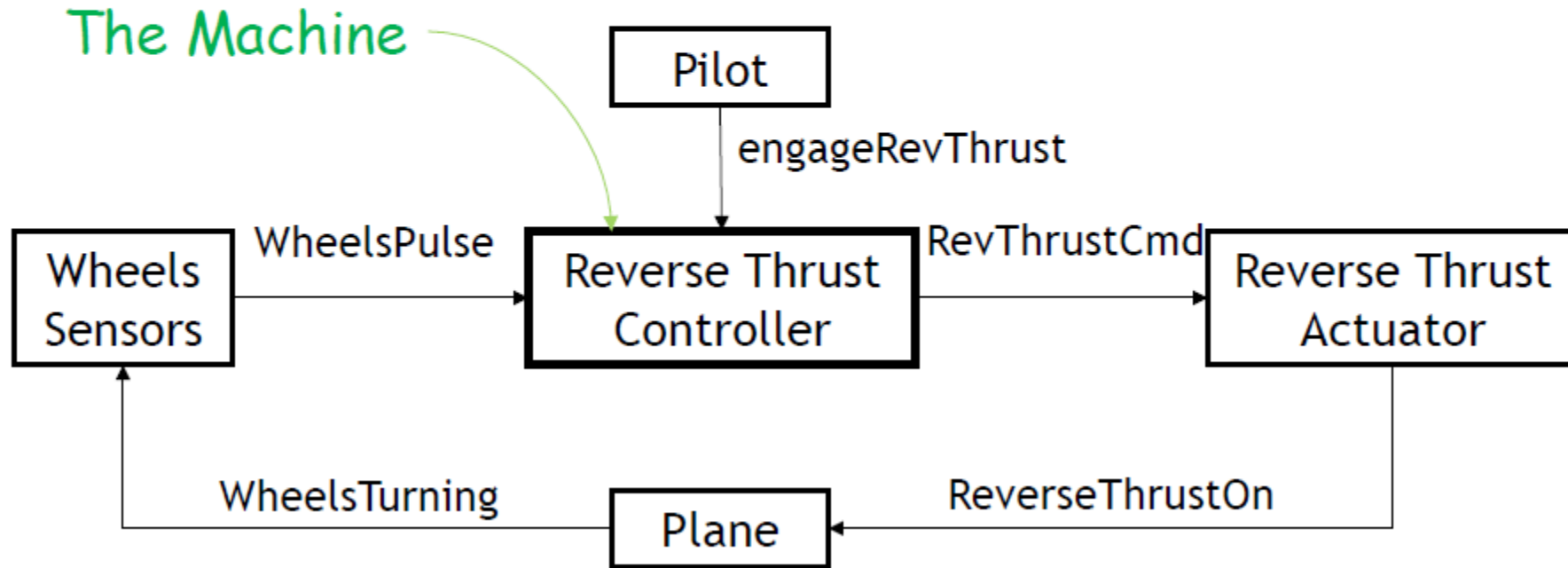


An interface where some phenomena are controlled by A and others by B

# Context Diagram for an Ambulance Dispatching System



## Context Diagram for an Airplane Reverse Thrust Controller

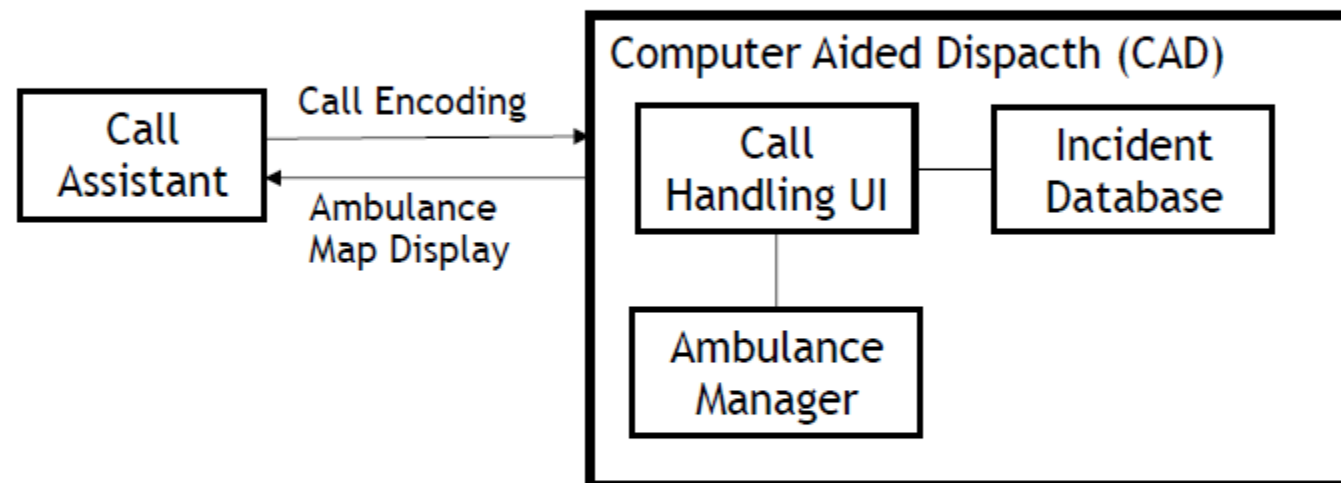


## DOs and DON'Ts of Context Diagrams

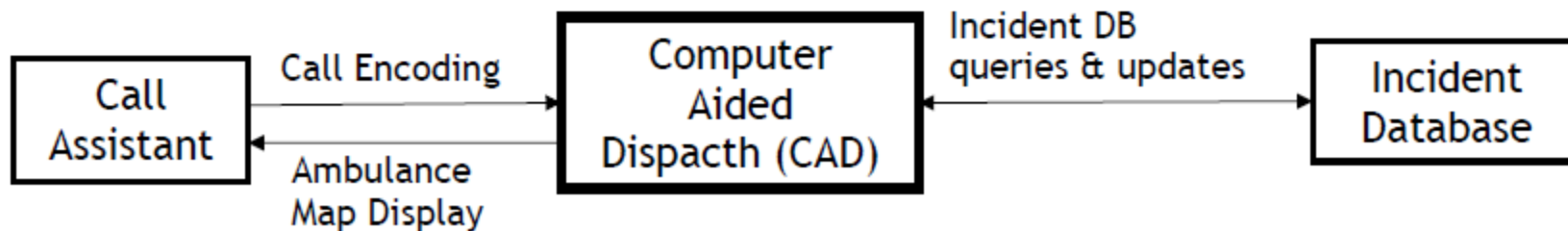


DON'T: model the Machine internal structure

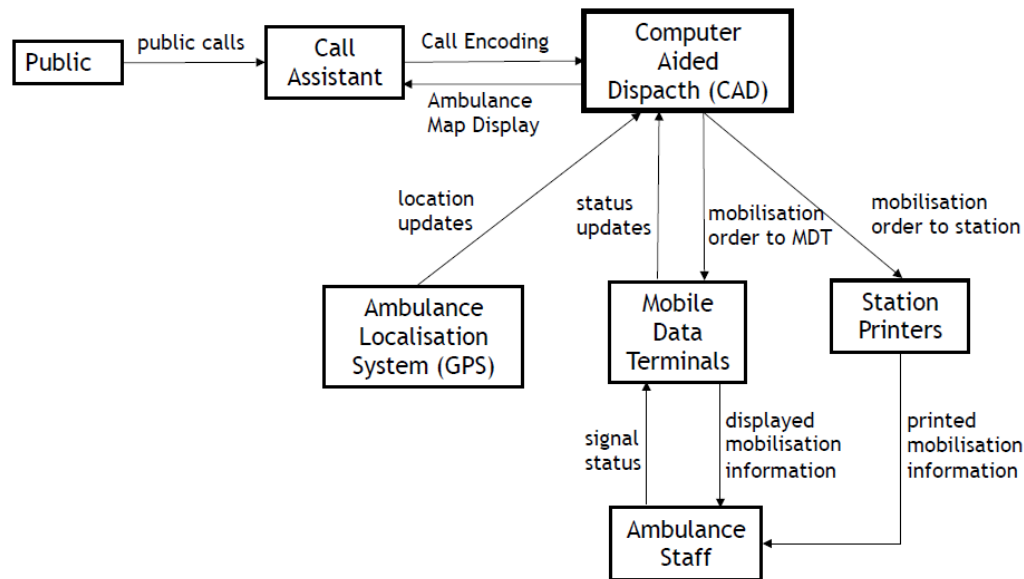
*Showing potential internal components is not needed*



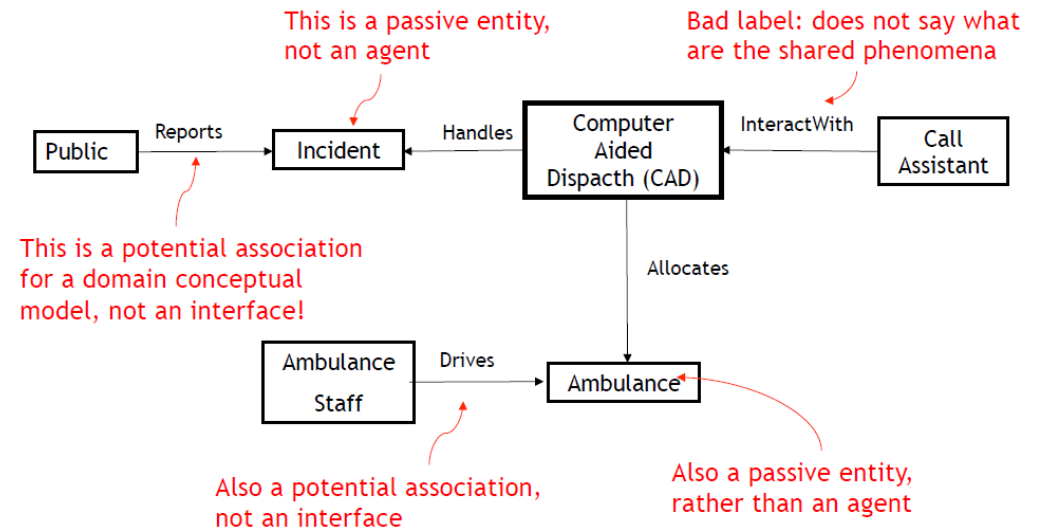
*Showing internal components as external agents is wrong*



## Context Diagram for an Ambulance Dispatching System

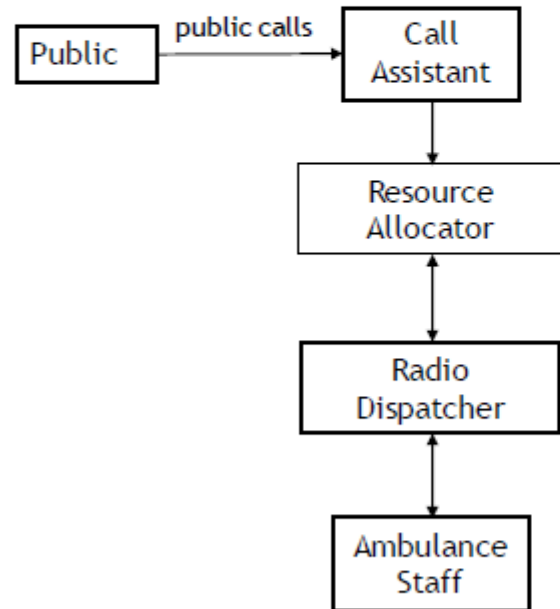


DON'T: try to force information that does not belong to a context diagram into a context diagram

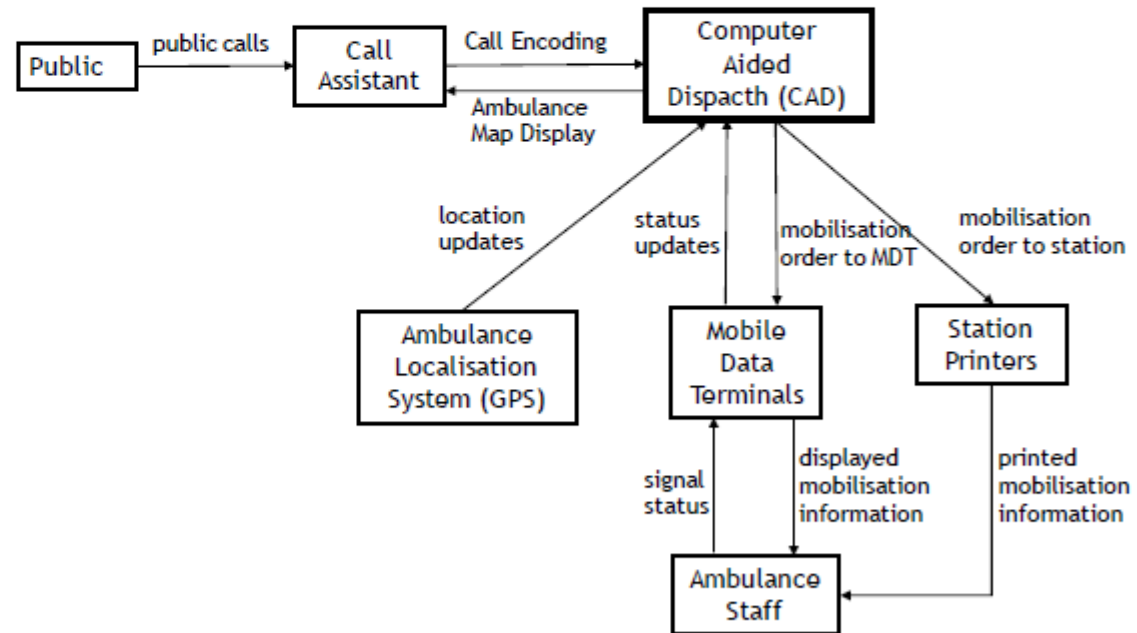


## DO: show Before and After Context Diagrams

Before  
(current system)



After  
(system-to-be)



# Exercise – System Boundary and Context Objects

Your university campus struggles with parking availability, especially during peak hours. Students, faculty, and staff often spend several minutes circling lots searching for available parking spots. To address this, the university plans to introduce a Campus Car-Parking App that provides real-time information about parking availability across all campus parking lots.

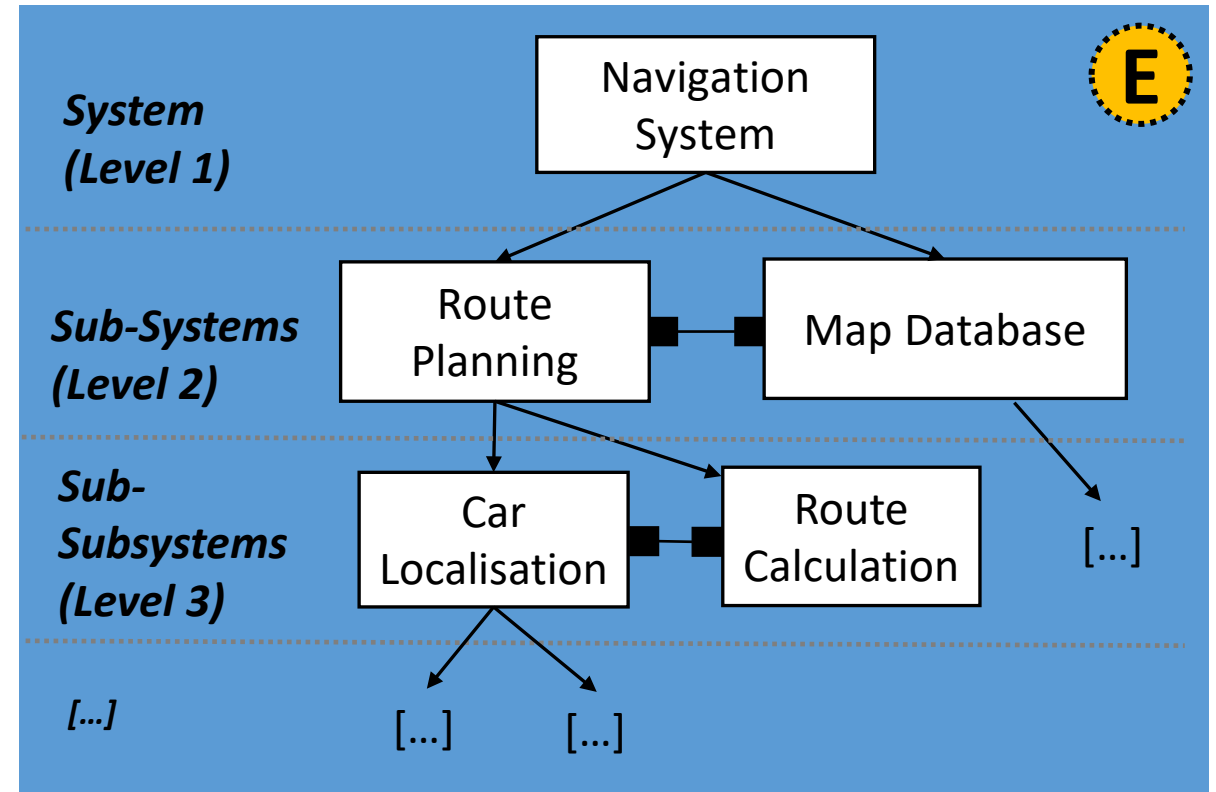
1. Write the system's goal in one sentence
2. Draw the system boundary and list the external actors and context objects
3. Label interactions (use arrows and annotations) between actors and system.
4. What is one issue that your group debated about placing inside vs. outside the boundary? Why?

Model answer to exercise will be populated  
after class

# System Decomposition and Context Consideration

## System Decomposition

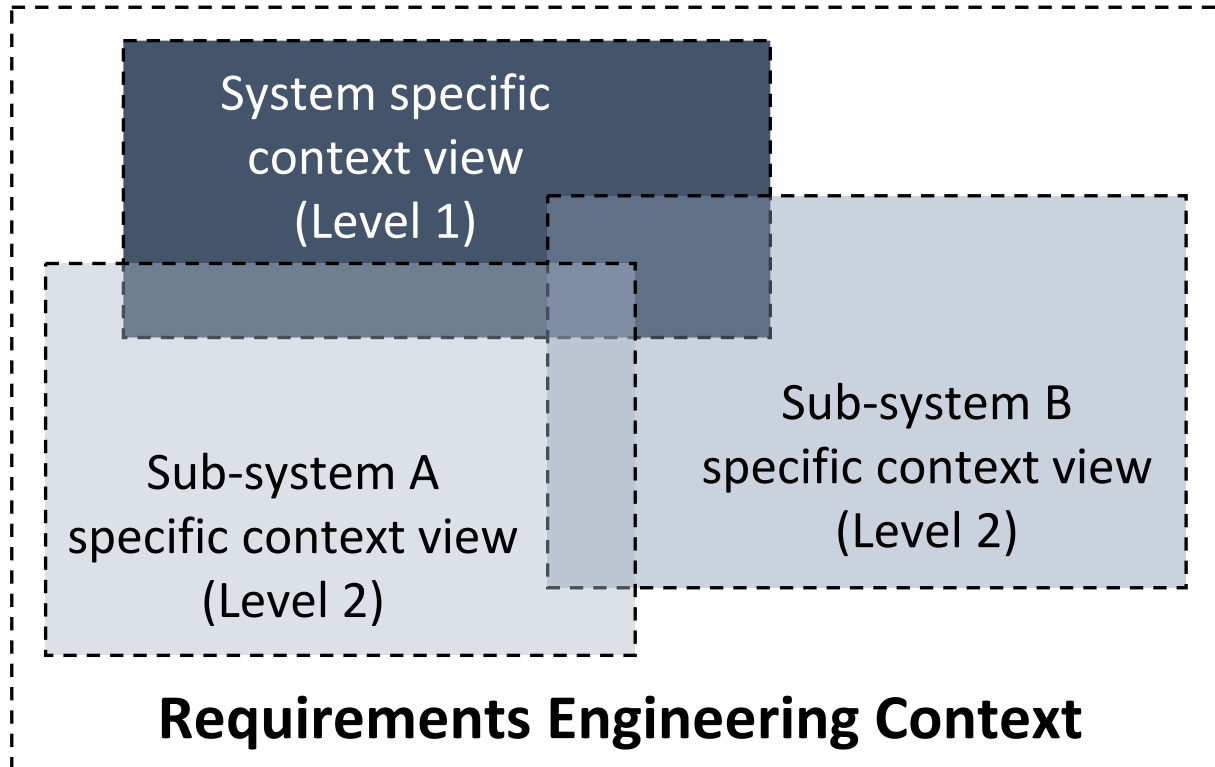
- The system is decomposed by defining an architecture, i.e., its constituent parts and their interfaces
- Number of abstraction levels depends on the complexity of the system
- Decomposition
  - Leads to different levels of abstraction (Sub-Systems, Sub-Subsystems, ...)
  - Reduces complexity
- Requirements defined at an abstraction level are assigned to the sub-systems at the abstraction level below [Whalen et al. 2013]



# Sub-System Specific Contexts (1)

- Each sub-system requires
  - A more detailed consideration of context objects and context information
  - The consideration of additional context objects and information
- Decide which context objects and context information should be considered for a given sub-system
- For a sub-system, other relevant sub-systems defined at the same level of abstraction are considered as context objects
- Overall, system decomposition leads to a more detailed and sub-system specific consideration of the requirements engineering context

## Sub-System Specific Contexts (2)



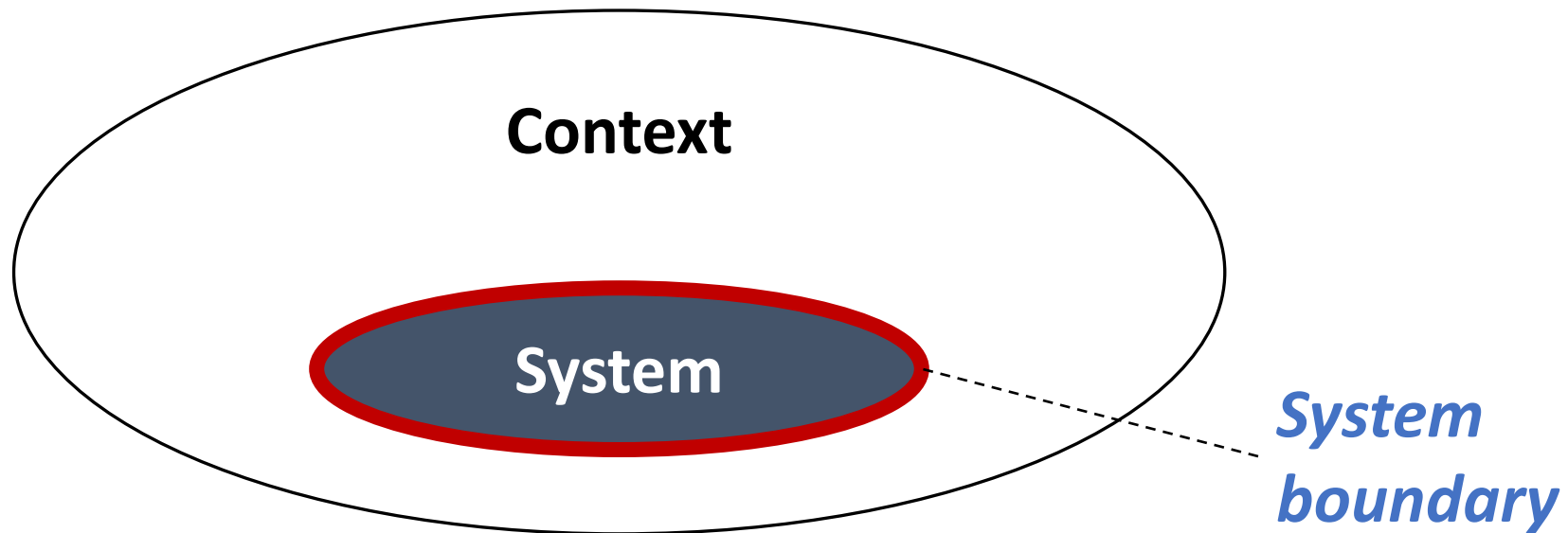
- System requirements defined at Level 1 are refined for the sub-systems defined at Level 2
- Context information and context objects considered
  - Differ at each level of abstraction
  - Differ between sub-systems



# System Boundary: Definition

**D** The system boundary is the “boundary between a system and its surrounding context”.

[Based on Glinz 2024, p. 27]



# Changeable and Unchangeable Objects

**Changeable objects**: Objects inside the system boundary can be changed (belong to the system)

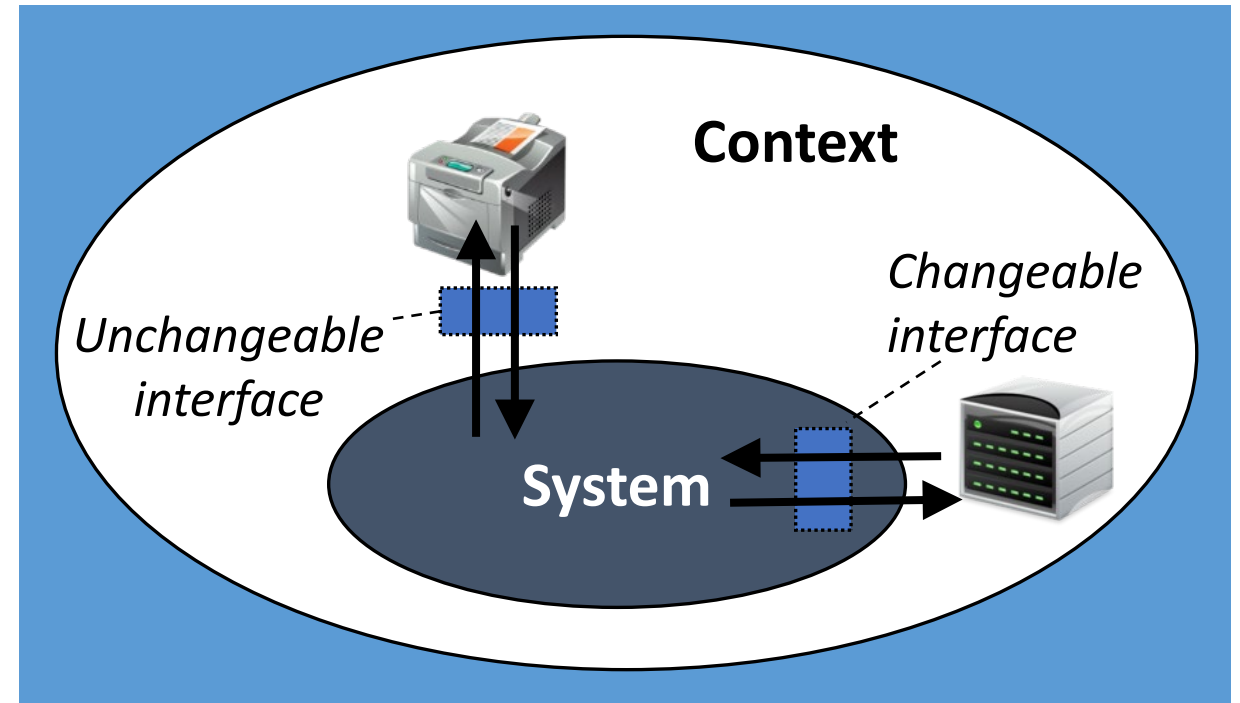
- E** The functionality to print documents shall be provided by the system. The stakeholders can decide which kind of printer will be used by the system. The printer is thus part of the system, i.e. within the system boundary.

**Unchangeable objects**: Objects outside the system boundary cannot be changed (belong to the context)

- E** An existing printer must be used for the system. Thus, the printer is a context object which cannot be changed.

# Changeable and Unchangeable Interfaces

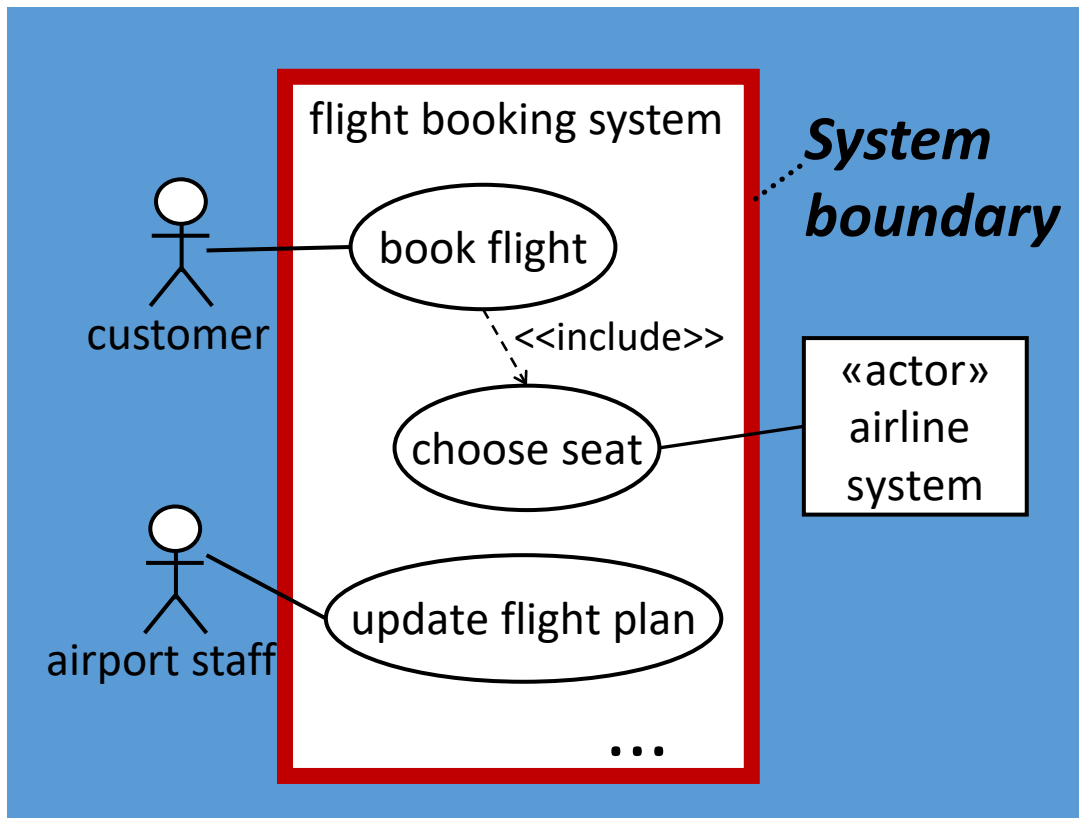
- Context objects interact with the system via interfaces
- Interfaces can be changeable (part of the system) or unchangeable (part of the context)
- Interfaces are thus first class objects (which in some cases be changed even if the related context objects cannot be changed)



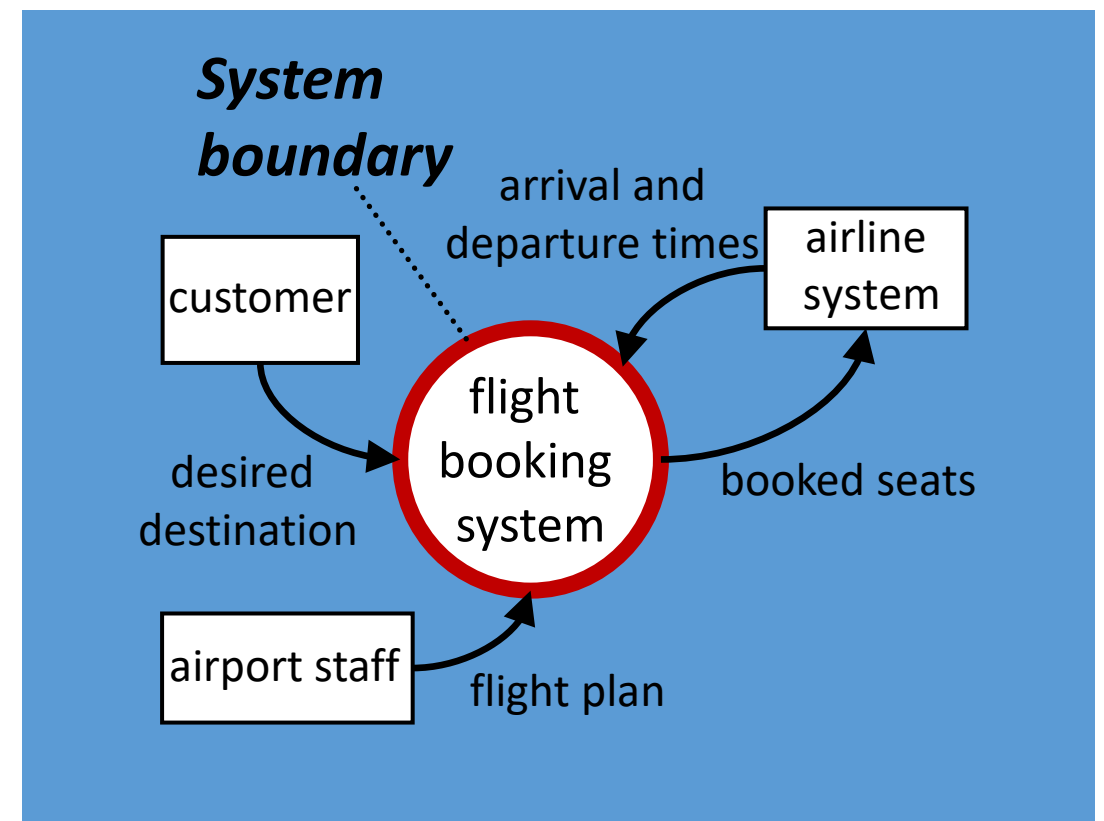
# System Boundaries in Requirements Models

E

- UML Use Case Diagram



## Data Flow Diagram

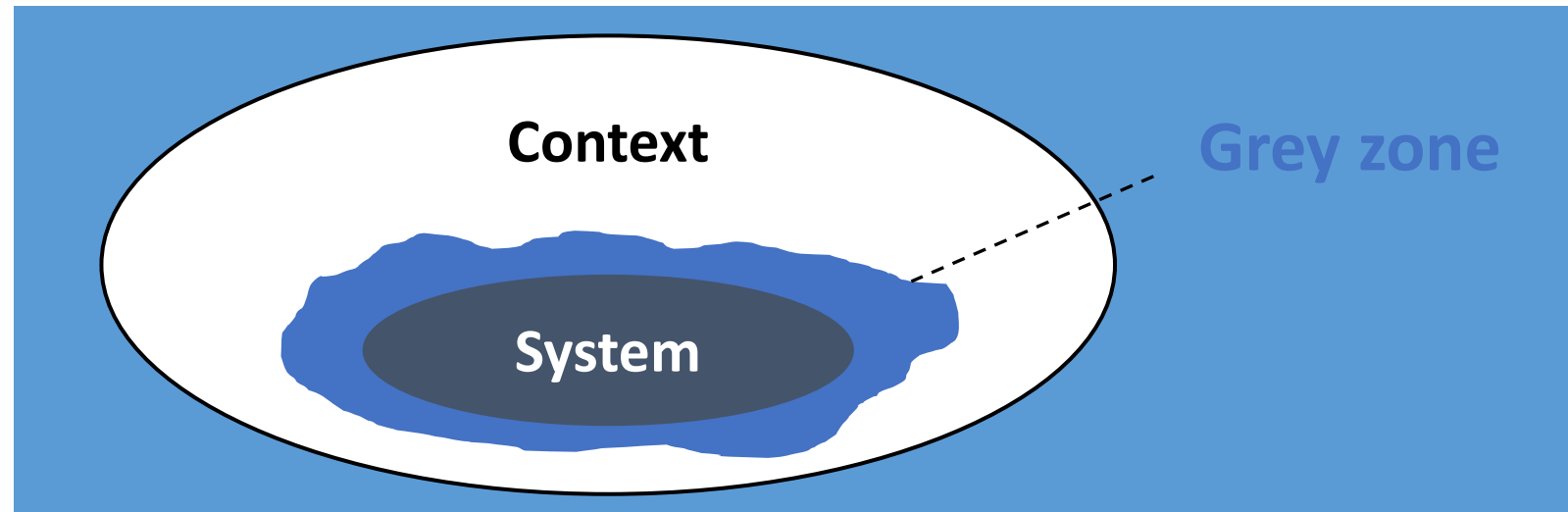


# Impact of System Boundary of Requirements

- An incorrect definition or consideration of the system boundary leads to wrong, missing or unnecessary requirements
- For each defined requirement, check the following:
  - Shall the (entire) requirement be realised by the system?
  - Can (part of) a requirement be satisfied by context objects, i.e., by reusing existing services, components or systems?
  - Is the requirement unnecessarily constrained by the system boundary, i.e., are objects considered as context objects, which should actually be changeable?

# Grey Zone

- Often it is unclear whether an object can be changed or not
- Grey zone covers objects (or parts of objects) for which it is not yet decided whether they can be changed or not
- Such (part of) objects should be explicitly classified as “grey zone objects”!



# Evolution

- The system boundary changes and evolves during system development (and operation), especially during early development phases
- Kinds of boundary changes:
  - Classification of previously unknown objects; new objects are classified as changeable or unchangeable
  - Changes in the classification of objects, i.e., a changeable object becomes unchangeable or vice versa
  - Changes in the classification of parts of an object, i.e., a changeable part of an object becomes unchangeable or vice versa
- Changes of the system boundary have impact on the requirements defined for the system; careful change integration is thus required

# System Boundary vs. Scope of Development

**D** The scope of development subsumes all “things that can be shaped and designed when developing a system”.

Based on [Glinz 2024, p.24]

- System boundary and scope of development are classifications with the same purpose
- Using both classifications unnecessarily increases complexity
- Use only one classification; preferably system boundary



## Perspective-Based Context Consideration

Example: Considering a Document from Different Perspectives

- The operation manual of a predecessor system contains, among other things, information about:
  - Usage perspective: Information about the user interfaces and the user-system interactions
  - Subject perspective: Information about the data recorded in the system, including the database schema
  - IT system perspective: Descriptions of the hard- and software environment of the predecessor system, interfaces, communication protocols etc.
  - Development perspective: Descriptions of allowed user-defined extensions to the system, as well as programming languages or compilers, which can be used

**Apply perspective-based reading to documents!**

# Summary

- A requirement is always defined for a particular context!
- The requirements engineering context:
  - Subsumes all context objects and context information relevant for requirements engineering
  - Has to be considered in all requirements engineering and all other development activities
- The context has a strong impact on:
  - The definition and interpretation of requirements
  - Feasible solution alternatives and thus restricts the solution space
- Each sub-system resulting from system decomposition has a sub-system-specific context (view)
- The system boundary separates changeable from unchangeable objects
  - Incorrectly classified objects can lead to incorrect, missing or unnecessary requirements
  - Grey zone for objects, which cannot be classified as changeable/unchangeable (yet)

## Literature References

- [Cambridge 2022] Cambridge Dictionary: English Meaning of Perspective.  
<https://dictionary.cambridge.org/dictionary/english/perspective> accessed on 15/05/2023.
- [Glinz 2024] M. Glinz: A Glossary of Requirements Engineering Terminology. Standard Glossary for the Certified Professional for Requirements Engineering (CPRE) Studies and Exam. Version 2.1.0 January 2024, 2024.
- [IREB 2022] IREB Certified Professional for Requirements Engineering, Foundation Level, Syllabus, Version 3.1.0, September 2022, [https://www.ireb.org/content/downloads/2-cpre-foundation-level-syllabus-3-0/cpre\\_foundationlevel\\_syllabus\\_en\\_v.3.1.pdf](https://www.ireb.org/content/downloads/2-cpre-foundation-level-syllabus-3-0/cpre_foundationlevel_syllabus_en_v.3.1.pdf); accessed on 15/05/2023.
- [Zave and Jackson 1997] P. Zave, M. Jackson: Four Dark Corners of Requirements Engineering. ACM ACM Transactions on Software Engineering and Methodology, Vol. 6, No. 1, 1997, pp. 1-30.

## Image References

- [1] Licensed by <http://www.icons shock.com/>
- [2] Provided by Microsoft Office

## Legend



Definition



Example